

Practical No.1: Write problem statement to define the project title with bounded scope of the project.

🔗 Project Title

Online Book Donation and Request Portal

🔗 Problem Statement

Access to educational and recreational books remains a significant challenge for underprivileged students, schools, and community libraries. Many individuals and institutions have surplus books that remain unused due to the absence of a structured platform for donation. On the other hand, organizations and students in need lack a convenient way to request and receive these resources.

Currently, the process of donating books is largely unorganized, depending on personal networks, NGOs, or local drives, which often leads to mismatch between donations and actual requirements. There is a need for a **centralized digital platform** that enables donors to list available books and allows seekers to request required books in a systematic, transparent, and efficient way.

The proposed **Online Book Donation and Request Portal** will act as a bridge between donors and receivers by providing an accessible, user-friendly, and secure system where:

- Donors can donate books by listing details.
 - Requesters can browse or search for available books.
 - The system matches requests with donations.
 - Admins can monitor and ensure fair and transparent transactions.
-

🔗 Scope of the Project (Bounded Scope)

The project will include the following functionalities within its **defined scope**:

1. **User Management**
 - Registration and login for donors and requesters.
 - Profile management.
2. **Book Donation Module**
 - Donors can add book details (title, author, category, condition).
 - Option to edit/delete donation entries.
3. **Book Request Module**
 - Requesters can search, filter, and request available books.
 - View status of requests.
4. **Matching & Allocation**
 - The system matches available donations with requests.
 - Notifications sent to donors and requesters.
5. **Admin Module**
 - Monitor donations and requests.
 - Approve or reject suspicious entries.
 - Generate reports on donation activity.

Out of Scope

- Real-time delivery/logistics of books.
 - Integration with third-party courier services.
 - Payment gateway (since it's donation-based).
-

📄 UML Use Case Diagram

Here's the **text-based description** of the UML diagram (I can also generate a visual diagram for you if needed):

Actors:

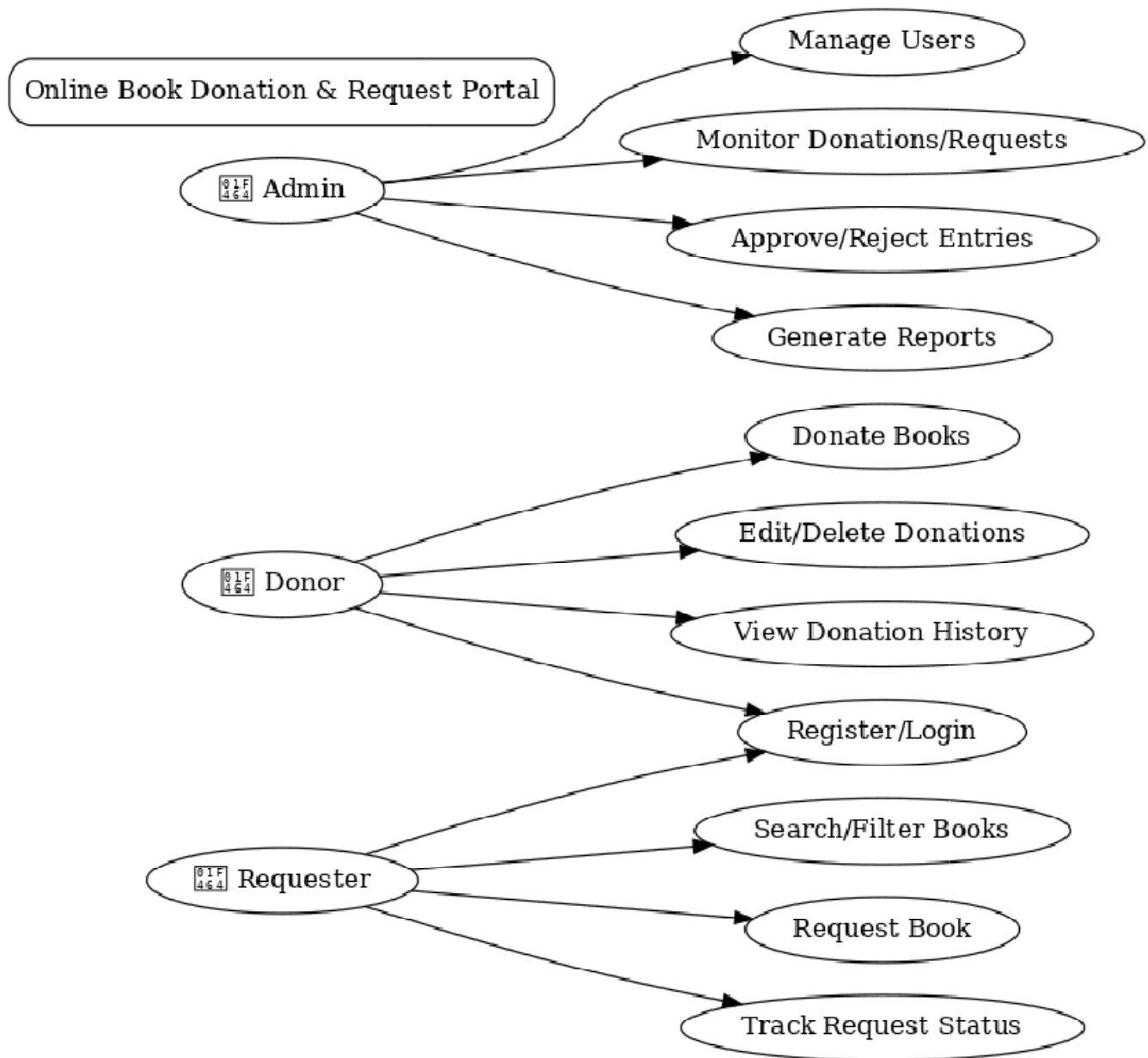
- **Donor**
- **Requester**
- **Admin**

Use Cases:

1. **Donor**
 - Register/Login
 - Donate Books (Add book details)
 - Edit/Delete donated books
 - View donation history
2. **Requester**
 - Register/Login
 - Search/Filter books
 - Request book
 - Track request status
3. **Admin**
 - Manage users (approve/reject)
 - Monitor books donated and requested
 - Approve/Reject suspicious donations/requests
 - Generate reports

Conclusion:

The **Online Book Donation and Request Portal** aims to create a sustainable and user-friendly digital solution to promote the reuse of books and foster a culture of sharing knowledge. With a defined scope and target audience, the platform is positioned to make a meaningful impact while remaining manageable and scalable for future enhancements.



Experiment :2 Process model to define activities and related tasks set

Process Model for Online Book Donation and Request Portal

1. Requirement Analysis

- Identify stakeholders (donors, requesters, admin)
- Gather requirements (donation, request, approval, notifications)
- Define functional & non-functional requirements
- Create use cases

2. System Design

- Design database (books, users, requests, transactions)
- Design UI/UX (donation form, request form, admin dashboard)
- Architecture design (3-tier: UI, application logic, database)
- Security design (authentication, data validation)

3. Implementation

- Develop donor module (book donation form, upload details)
- Develop requester module (search/request books)
- Admin module (approve, track, manage inventory)
- Integration of modules (frontend-backend)

4. Testing

- Unit testing (forms, database functions)
- Integration testing (donation–request workflow)
- System testing (full functionality check)
- User acceptance testing (with stakeholders)

5. Deployment

- Deploy portal on web server
- Configure database
- Provide user manuals/training

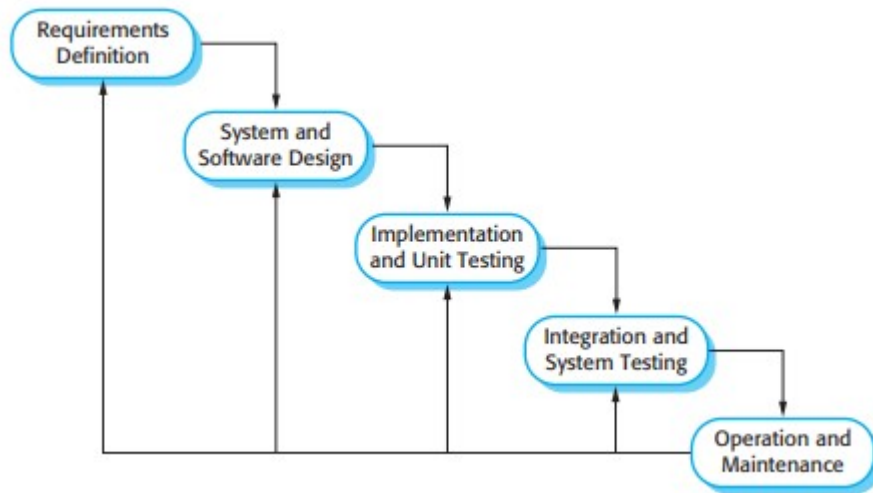
6. Maintenance & Support

- Monitor system performance
- Fix bugs and security issues
- Provide upgrades (new features, scalability)

Process Flow Diagram (represent in graphical)

Requirement Analysis → System Design → Implementation → Testing → Deployment → Maintenance

Waterfall Model Phases



1. Requirement Analysis

- Collect all requirements from stakeholders.
- Document functional & non-functional needs.

2. System Design

- Convert requirements into architecture & design.
- Create database schema, UI design, and overall system structure.

3. Implementation (Coding)

- Developers write code according to the design.
- Modules are developed and integrated.

4. Testing

- Perform unit, integration, and system testing.
- Ensure system works as per requirements.

5. Deployment

- Install and launch the software for end-users.
- Configure environment & train users.

6. Maintenance

- Fix bugs, apply updates, and improve performance.
- Provide long-term support.

Practical No.3: Gather application specific requirements for assimilate into RE (Requirement's engineering) Model

Application-Specific Requirements for RE Model

1. User Requirements

- Users should be able to register and log in to the portal.
 - Users can create a profile with basic details (name, email, contact, location).
 - Donors should be able to list books for donation.
 - Requesters should be able to search and request books.
 - Users can track the status of their donation/request.
 - Notifications (email/SMS/in-app) should be sent for donation confirmations and book availability.
 - Users can give feedback or rate the transaction.
-

2. Functional Requirements (FR)

1. **User Management**
 - Registration, login, logout, password reset.
 - Role-based access (Admin, Donor, Requester).
 2. **Book Management**
 - Donor can add books with title, author, category, condition, pickup/delivery preference.
 - Requester can search books by title, author, category, or location.
 - Book status management (Available, Reserved, Donated).
 3. **Donation/Request Process**
 - Requester sends a request for a book.
 - Donor accepts/rejects request.
 - Portal updates status and sends notifications.
 4. **Admin Features**
 - Monitor user activity and approve new donors.
 - Manage book categories and metadata.
 - Generate reports (number of donations, active users, most requested books).
 5. **Communication**
 - Secure in-app messaging between donor and requester.
 - Email/SMS alerts for important events.
-

3. Non-Functional Requirements (NFR)

- **Performance:** The system should handle at least 1000 concurrent users.
- **Usability:** User-friendly UI with simple navigation.
- **Security:**
 - Encrypt sensitive user data.
 - Verify donor/requester authenticity.
 - Prevent fake accounts (CAPTCHA, email verification).
- **Scalability:** The system should support expansion to multiple cities.

- **Reliability:** 99.5% uptime.
 - **Portability:** Responsive design for desktop, tablet, and mobile.
-

4. System Requirements

- **Hardware:**
 - Server with minimum 8GB RAM, Quad-core processor, 100 GB storage.
 - Internet connection with 1Gbps bandwidth for hosting.
 - **Software:**
 - Backend: Django / Node.js / PHP
 - Database: MySQL / PostgreSQL
 - Frontend: HTML, CSS, JavaScript (React/Angular optional)
 - Notifications: Email/SMS API
-

5. Constraints

- Books should not be sold — only donation and request allowed.
 - Delivery method (pickup by requester or courier) should be clarified by donor.
 - Limited book categories at the start (expandable later).
-

6. Assumptions

- Users have internet access and basic digital literacy.
 - Books donated are in usable condition.
 - Donors are willing to share location/contact for pickup.
-

1. Requirement Specification Table (SRS Extract)

Req ID	Requirement Type	Description	Priority
UR-01	User Requirement	User shall be able to register, login, and manage their profile.	High
UR-02	User Requirement	Donors shall be able to donate books by listing details.	High
UR-03	User Requirement	Requesters shall be able to search and request books.	High
FR-01	Functional	System shall allow role-based access (Admin, Donor, Requester).	High
FR-02	Functional	System shall allow donors to update book status (Available, Reserved, Donated).	Medium
FR-03	Functional	System shall notify users about requests, approvals, and donations.	High
FR-04	Functional	Admin shall be able to monitor users and generate donation reports.	Medium
NFR-01	Non-Functional	The system shall be available 99.5% of the time.	High
NFR-02	Non-Functional	The system shall ensure secure authentication and data encryption.	High
NFR-03	Non-Functional	The system shall support at least 1000 concurrent users.	Medium
SYS-01	System	The system shall run on a web server with 8GB RAM and MySQL/PostgreSQL DB.	High

2. Use Case Diagram (UML)

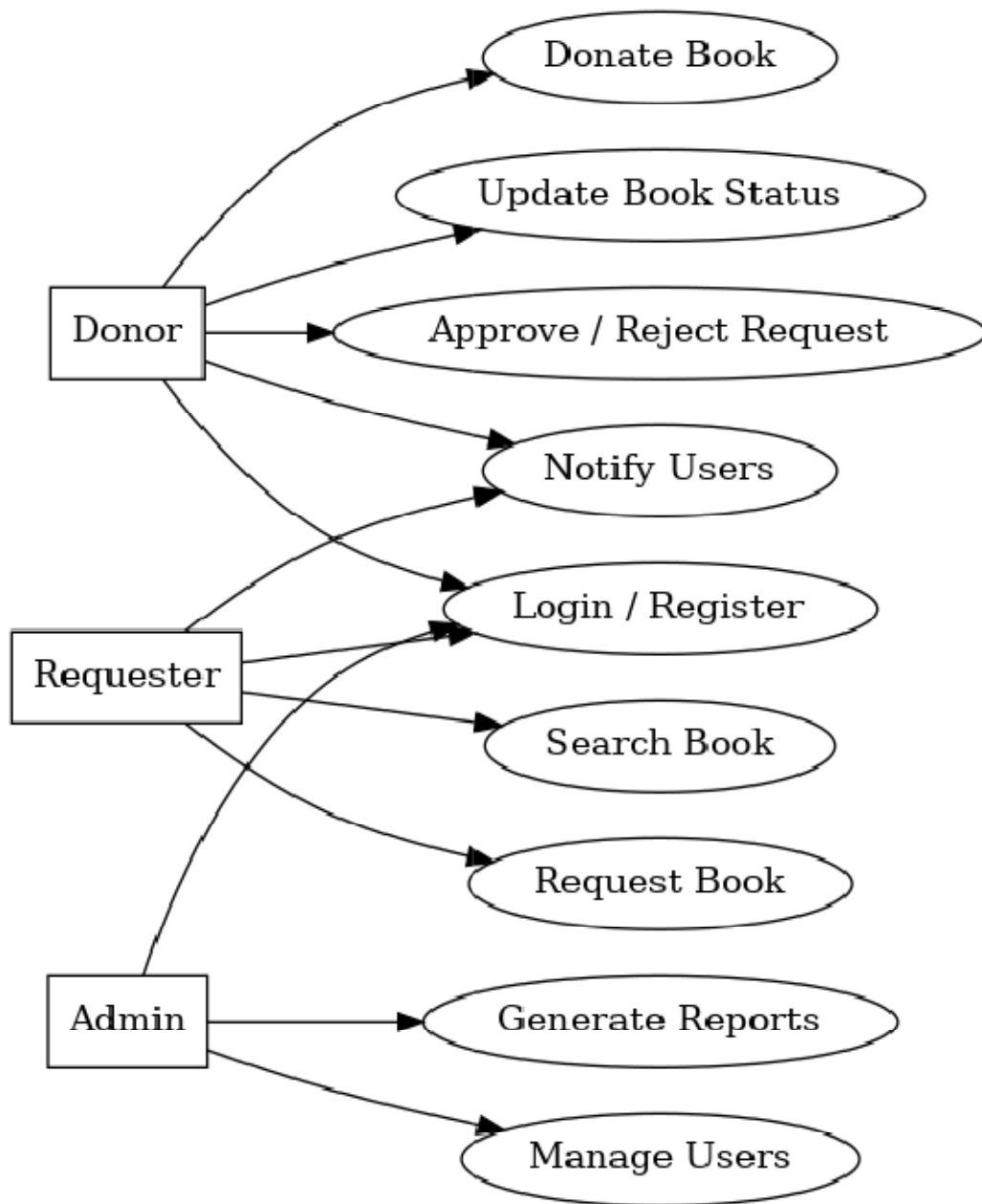
Actors:

- **Admin**
- **Donor**
- **Requester**

Main Use Cases:

- Register / Login
- Manage Profile
- Donate Book
- Search Book

- Request Book
- Approve / Reject Request (Donor)
- Notification Handling
- Admin Monitoring & Reports



Practical No.4: Prepare broad SRS (software requirement Specification) for the project.

Software Requirement Specification (SRS)

Project Title: Online Book Donation and Request Portal

1. Introduction

1.1 Purpose

The purpose of this portal is to provide a centralized platform for individuals to donate books and for others to request them. The system eliminates barriers between donors and requesters, promoting book reusability, literacy, and community support.

1.2 Scope

The system will:

- Allow **donors** to list books for donation.
- Allow **requesters** to search and request books.
- Provide **administrative control** to monitor donations and generate reports.
- Support secure user registration, communication, and notifications.
- Be scalable to multiple locations and categories.

Out of scope: Selling or auctioning books, digital e-book distribution.

1.3 Definitions, Acronyms, Abbreviations

- **Donor:** A user who donates books.
- **Requester:** A user who searches and requests books.
- **Admin:** System manager overseeing portal activities.
- **SRS:** Software Requirement Specification.

1.4 References

- IEEE 830 Standard for SRS Documentation.
 - UML diagrams (Use Case, Process Flow).
-

2. Overall Description

2.1 Product Perspective

The system is a web-based application, designed with a client-server architecture. Users interact via web browsers, while backend processes handle authentication, data management, and notifications.

2.2 Product Functions

- User registration and login.
- Book donation management (add, edit, update status).
- Book request and approval process.
- Notifications (email/SMS/in-app).
- Reporting and monitoring for admins.

2.3 User Classes and Characteristics

- **Donor:** Tech-savvy individuals with internet access, willing to donate books.
- **Requester:** General users seeking books.
- **Admin:** Technical staff responsible for system monitoring and reporting.

2.4 Operating Environment

- **Frontend:** Web browser (Chrome, Firefox, Edge).
- **Backend:** Django / Node.js / PHP.
- **Database:** MySQL / PostgreSQL.
- **Platform:** Cloud-hosted web server with scalability options.

2.5 Design and Implementation Constraints

- Must follow **open-source license compliance**.
- Must support responsive design (mobile-friendly).
- Must ensure **data privacy (GDPR compliance if expanded internationally)**.

2.6 Assumptions and Dependencies

- Users have stable internet access.
 - Donors provide accurate book condition information.
 - Third-party email/SMS APIs are available for notifications.
-

3. Specific Requirements

3.1 Functional Requirements (FR)

FR-01: System shall allow users to register, login, and manage profiles.
FR-02: Donors shall be able to donate books with details (title, author, condition, category).
FR-03: Requesters shall be able to search and request books.
FR-04: Donors shall approve or reject book requests.
FR-05: System shall update book status (Available, Reserved, Donated).
FR-06: Notifications shall be sent on major events (donation approved, book available).
FR-07: Admin shall manage users and monitor donation trends.
FR-08: Admin shall generate reports (donation count, top categories, active users).

3.2 Non-Functional Requirements (NFR)

NFR-01: The system shall support at least 1000 concurrent users.
NFR-02: The system shall ensure **99.5% uptime**.
NFR-03: Sensitive user data shall be encrypted.
NFR-04: The system shall be mobile-friendly and accessible (WCAG compliance).
NFR-05: System shall provide responses to user actions within **3 seconds**.

3.3 System Requirements

- **Hardware:** 8GB RAM, Quad-core CPU, 100GB storage server.
 - **Software:** Django/Node.js backend, MySQL/PostgreSQL DB, Email/SMS API integration.
 - **OS:** Linux-based web server.
-

4. External Interface Requirements

4.1 User Interface (UI)

- Clean, intuitive interface with simple navigation.
- Responsive design for desktops, tablets, and mobiles.
- Search bar, book listing grid, and donation form.

4.2 Hardware Interfaces

- Web server hosting environment.
- Client systems with standard browsers.

4.3 Software Interfaces

- Database system (MySQL/PostgreSQL).
- Email/SMS API for notifications.
- Authentication service (JWT / OAuth).

4.4 Communication Interfaces

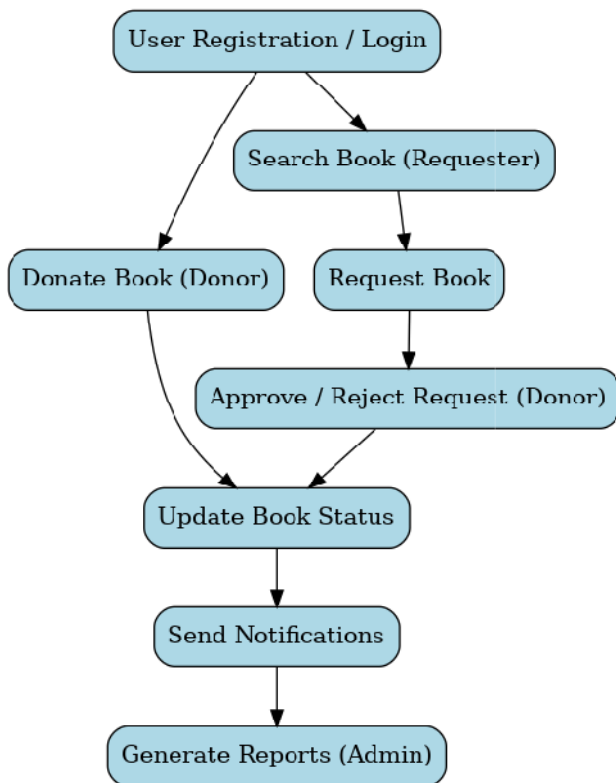
- HTTPS protocol for secure communication.
 - REST API for integration with future mobile apps.
-

5. System Models

- **Use Case Diagram:** (Already prepared).
 - **Process Flow Diagram:** (Register → Donate/Request → Approval → Notification → Report).
-

6. Appendix

- Initial deployment limited to a single city; later scalable.
- Community feedback module may be added in future releases.



Practical No.5

Title: Use Cases and Use Case Diagram for Online Book Donation and Request Portal

Use Cases

Actors

- **Donor** – Donates books, manages status, approves requests.
 - **Requester** – Searches and requests books.
 - **Admin** – Manages users, monitors activity, generates reports.
-

Use Case Descriptions

1. Register/Login

- **Actor:** Donor, Requester, Admin
- **Description:** Users must register and log in to access system features.
- **Precondition:** User has internet access.
- **Postcondition:** User gains access to role-specific features.

2. Donate Book

- **Actor:** Donor
- **Description:** Donor lists book details (title, author, condition, category, availability).
- **Precondition:** Donor must be logged in.
- **Postcondition:** Book is added to the available list.

3. Search Book

- **Actor:** Requester
- **Description:** Requester searches books using filters (title, author, category, location).
- **Precondition:** Requester must be logged in.
- **Postcondition:** Matching books are displayed.

4. Request Book

- **Actor:** Requester
- **Description:** Requester sends a request for an available book.
- **Precondition:** Book must be listed as available.
- **Postcondition:** Request is sent to the donor.

5. Approve/Reject Request

- **Actor:** Donor

- **Description:** Donor approves or rejects a requester's book request.
- **Precondition:** Book must be available.
- **Postcondition:** Book status updated (Reserved/Rejected).

6. Update Book Status

- **Actor:** Donor
- **Description:** Donor updates book status (Available, Reserved, Donated).
- **Precondition:** Book must exist in system.
- **Postcondition:** Status updated in database.

7. Notifications

- **Actor:** Donor, Requester
- **Description:** System sends notifications about approvals, donations, and status updates.
- **Precondition:** Request or donation event occurs.
- **Postcondition:** Users are notified.

8. Manage Users

- **Actor:** Admin
- **Description:** Admin monitors user activity and manages accounts.
- **Precondition:** Admin logged in.
- **Postcondition:** User access is maintained or revoked.

9. Generate Reports

- **Actor:** Admin
- **Description:** Admin generates reports of donations, requests, and active users.
- **Precondition:** Admin logged in.
- **Postcondition:** Reports are available for review.

Use Case Diagram

Actors:

- Donor
- Requester
- Admin

Main Use Cases:

- Register/Login
- Donate Book
- Search Book
- Request Book
- Approve/Reject Request
- Update Book Status
- Notifications
- Manage Users
- Generate Reports

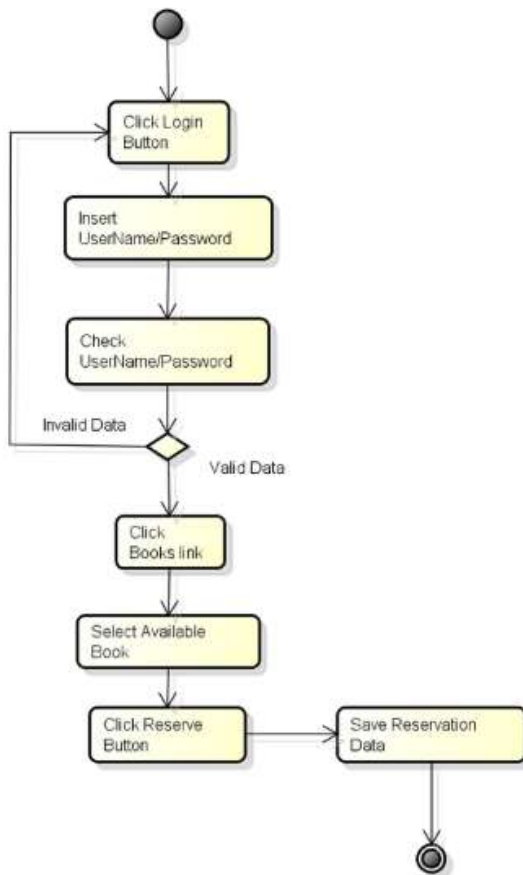
Draw USE case diagram using StarUML

Practical No.6: Draw the activity diagram to represent flow from one activity to another for software development.

Activity Diagram – Online Book Donation and Request Portal

Flow of Activities

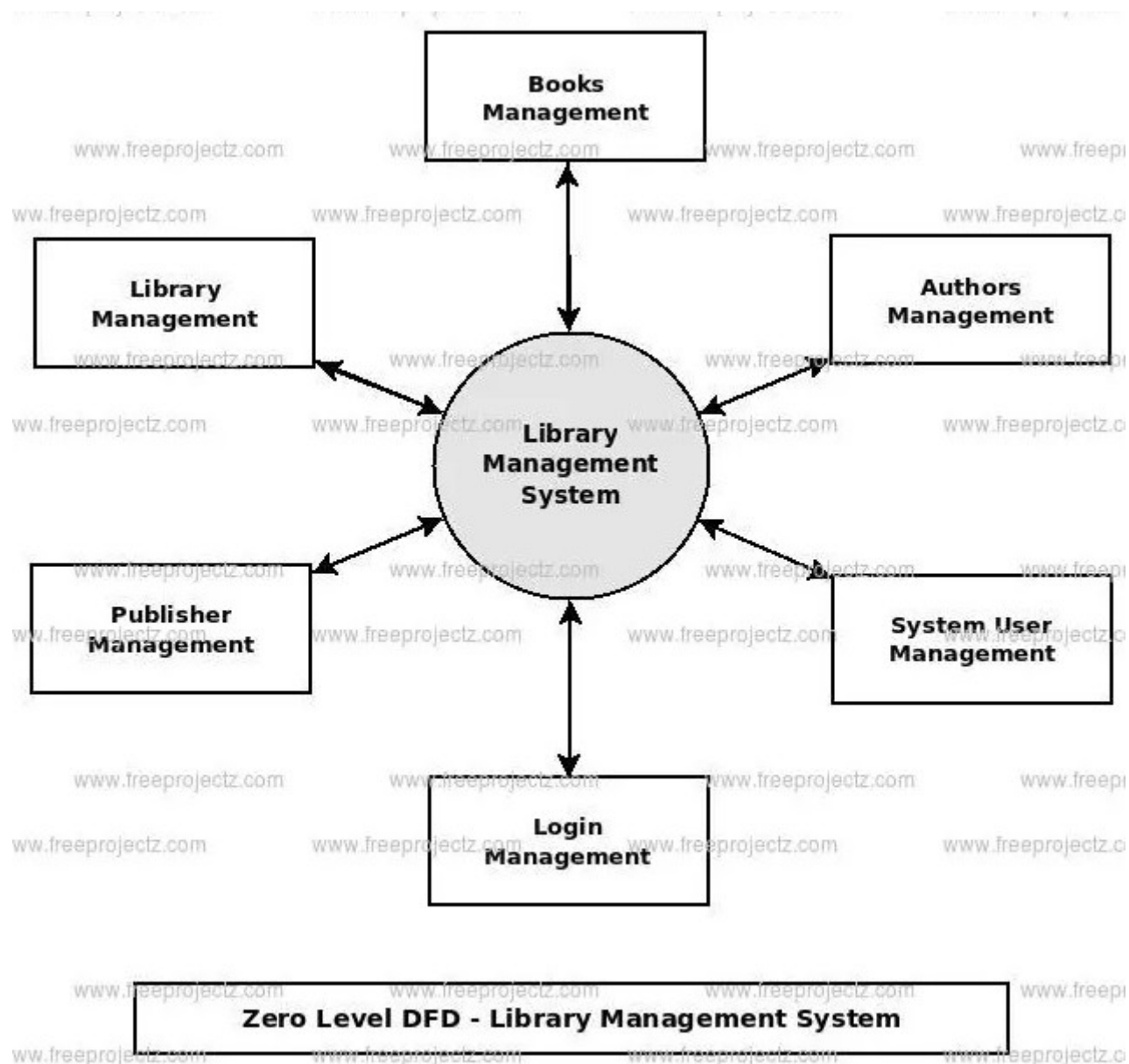
Activity diagram for online book donation system

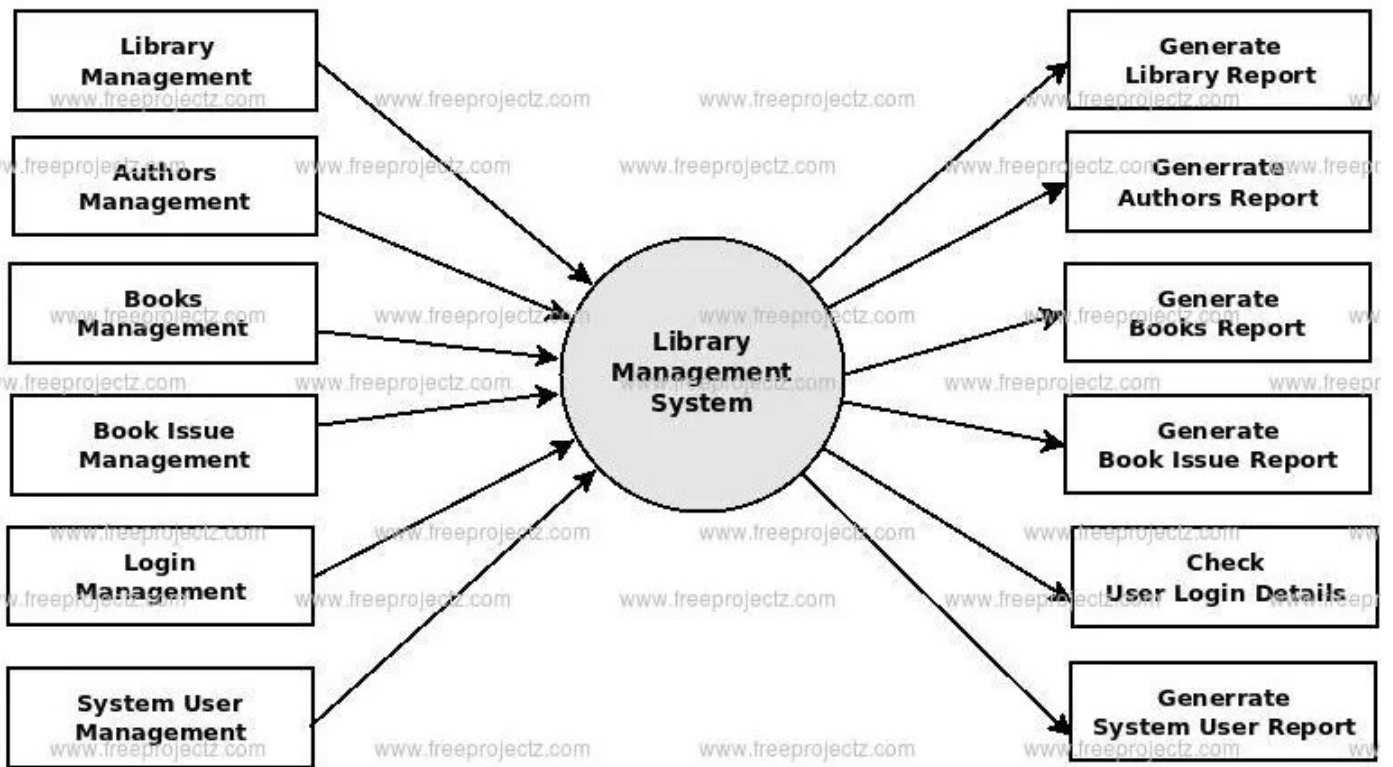


Experiment 7 : *Create DFDs(data flow diagram), Decision tables and E-R (entity-relationship) diagram

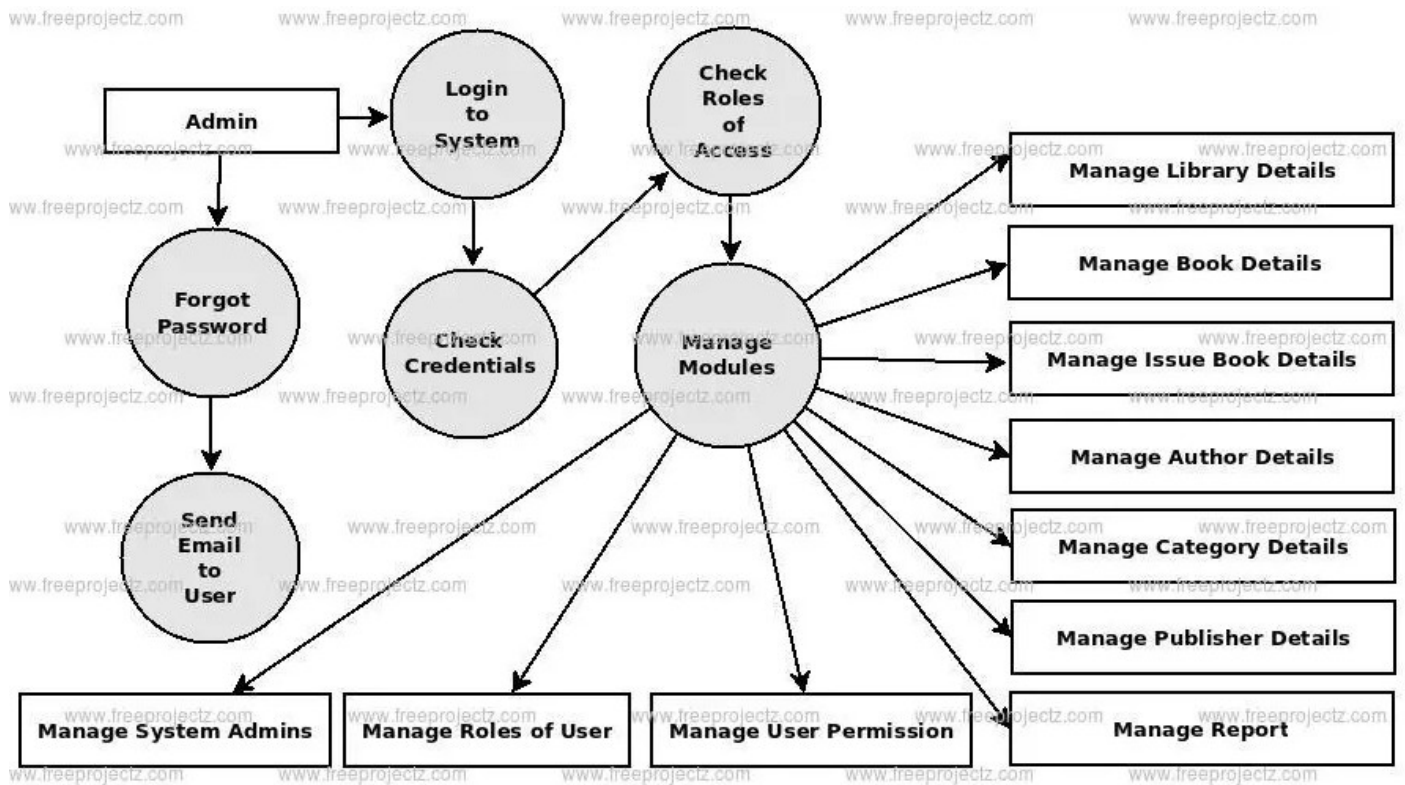
ER Diagram for Library Management system

Draw yourself





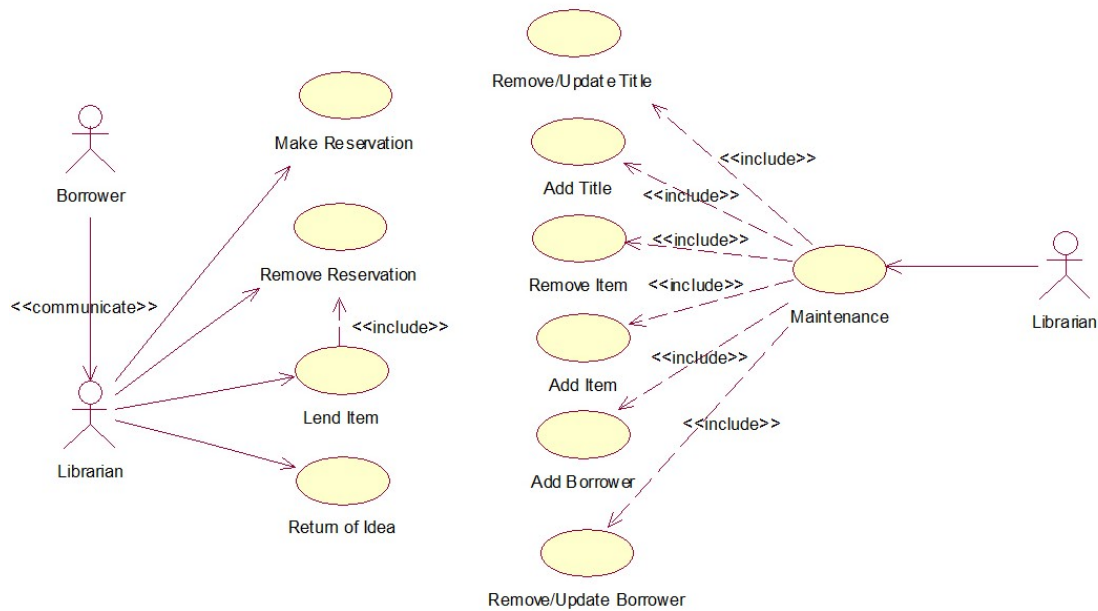
First Level DFD - Library Management System



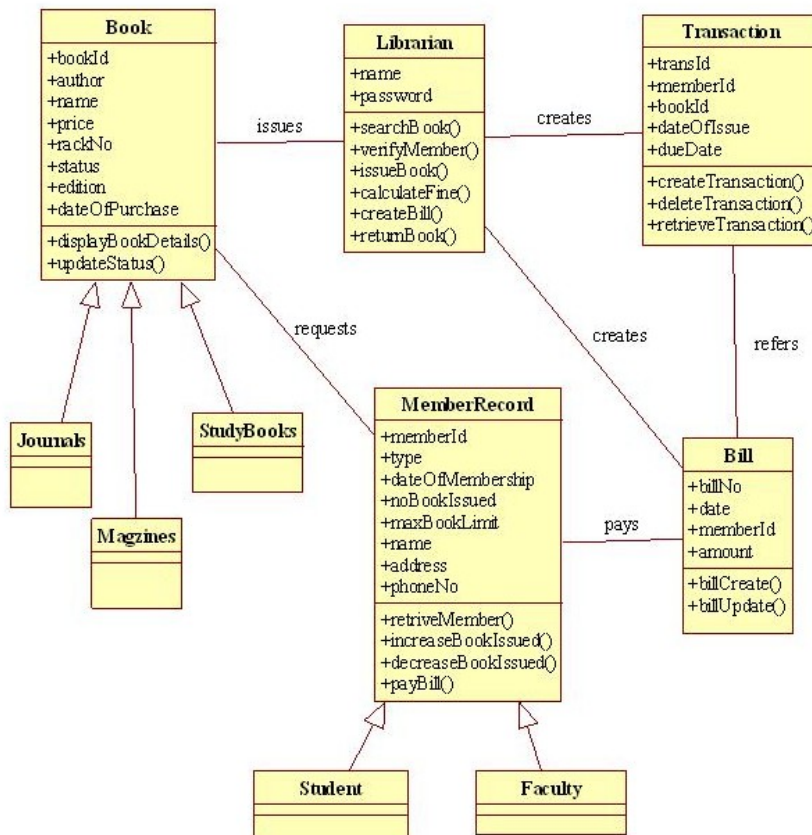
Second Level DFD - Library Management System

Experiment 8: Draw class diagram and Sequence diagram, State Transition Diagram

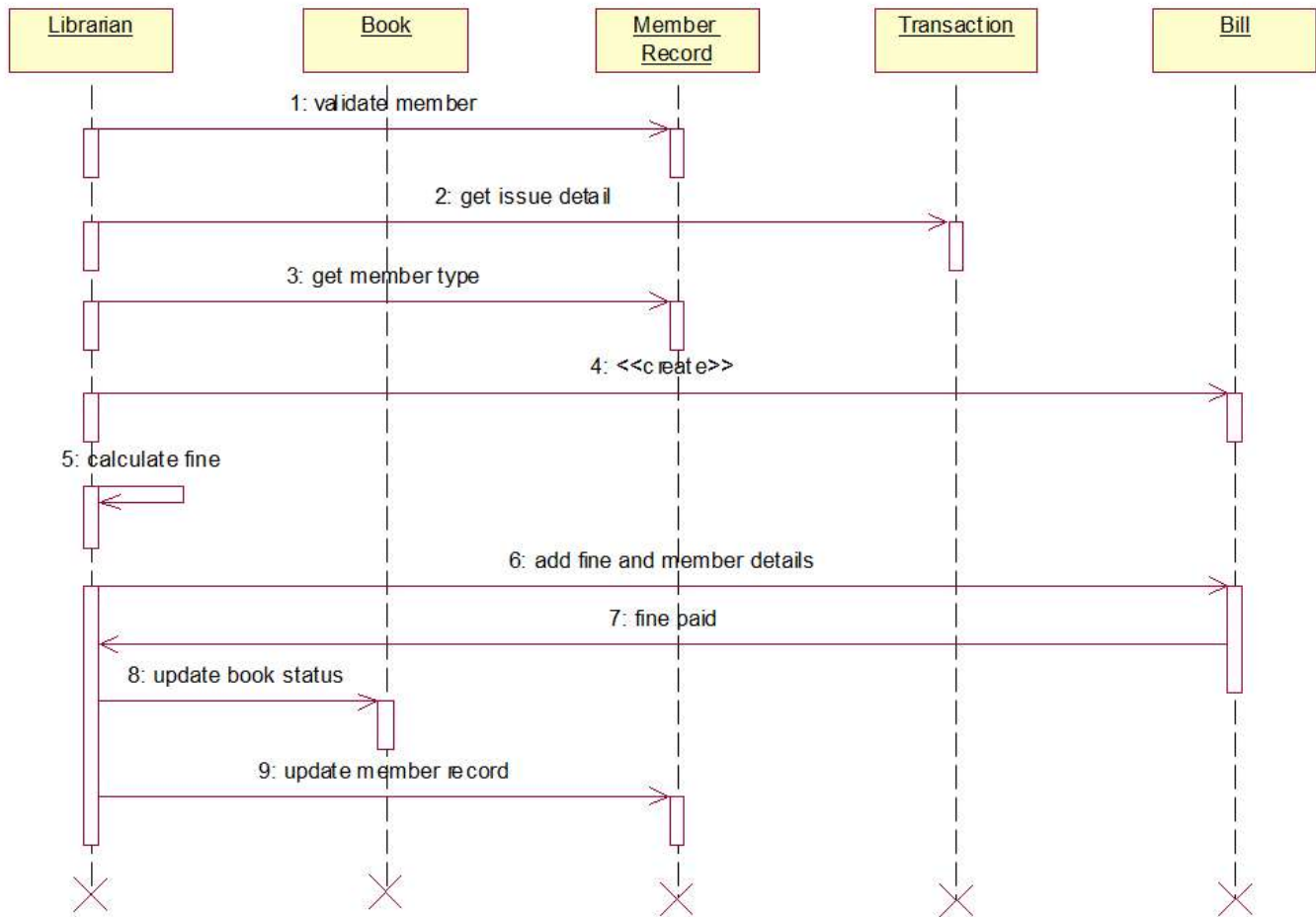
a). USECASE Diagram



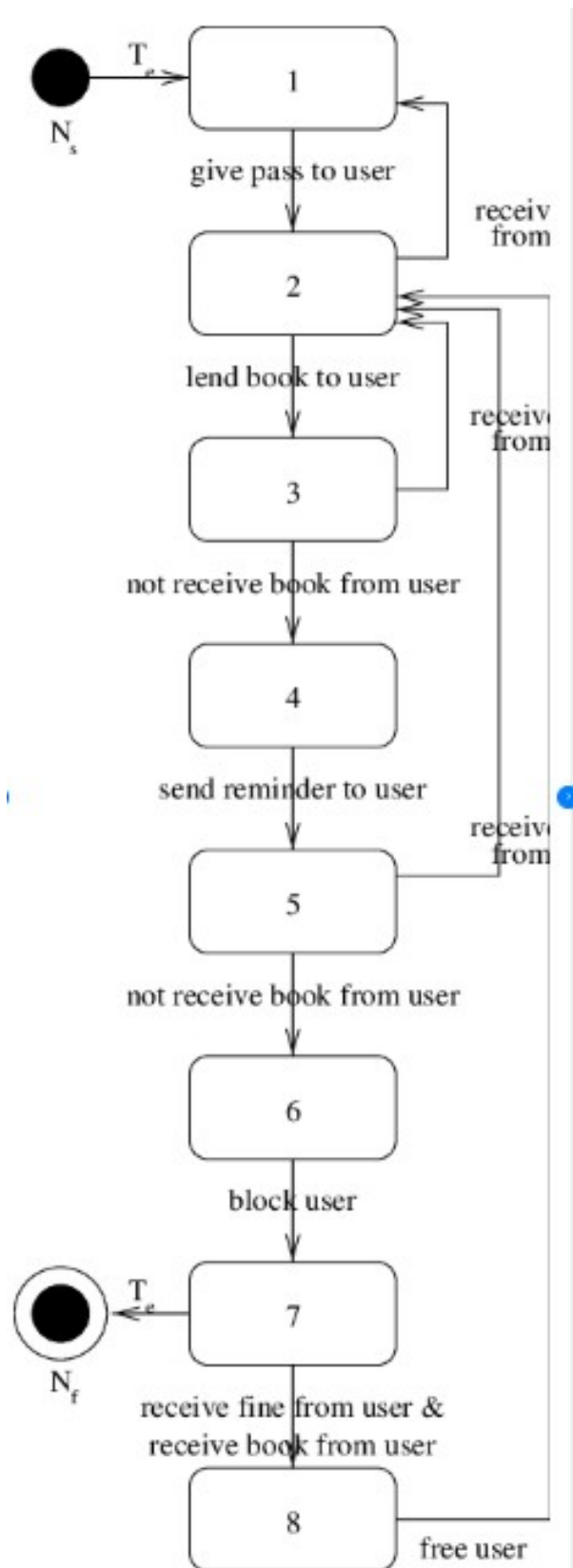
Class Diagram



Sequence Diagram



State Transition Diagram for Library management system



Experiment 9: Create decision table for a project.

i). Decision Table for Library management System

Conditions				
Valid selection	No	Yes	Yes	Yes
New member	-	Yes	No	No
Renewal	-	No	Yes	No
Cancellation	-	No	No	Yes
Actions				
Display error message	x	-	-	-
Ask member's details	-	x	-	-
Build customer record	-	x	-	-
Generate bill	-	x	x	-
Ask member's name & membership number	-	-	x	x
Update expiry date	-	-	x	-
Print cheque	-	-	-	x
Delete record	-	-	-	x

Experiment 10: Write test cases to validate requirements from SRS document.

Test Cases For Library Management System

LOGIN FORM:

SL.No	Test Case	Excepted Result	Test Result
1	Enter valid name and password & click on login button	Software should display main window	Successful
2	Enter invalid	Software should not display main window	successful

BOOK ENTRY FORM:

SL.No	Test Case	Excepted Result	Test Result
1	On the click of ADD button	At first user have to fill all fields with proper data , if any Error like entering text data instead of number or entering number instead of text..is found then it gives proper message otherwise Adds Record To the Database	successful
2.	On the Click of DELETE Button	This deletes the details of book by using Accession no.	Successful
3.	On the Click of UPDATE Button	Modified records are Updated in database by clicking UPDATE button.	Successful
4.	On the Click of SEARCH Button	Displays the Details of book for entered Accession no. Otherwise gives proper Error message.	Successful
5.	On the Click of CLEAR Button	Clears all fields	Successful
6.	On the Click of EXIT button	Exit the current book details form	successful
7.	On the Click of NEXT button	Display the next form	successful

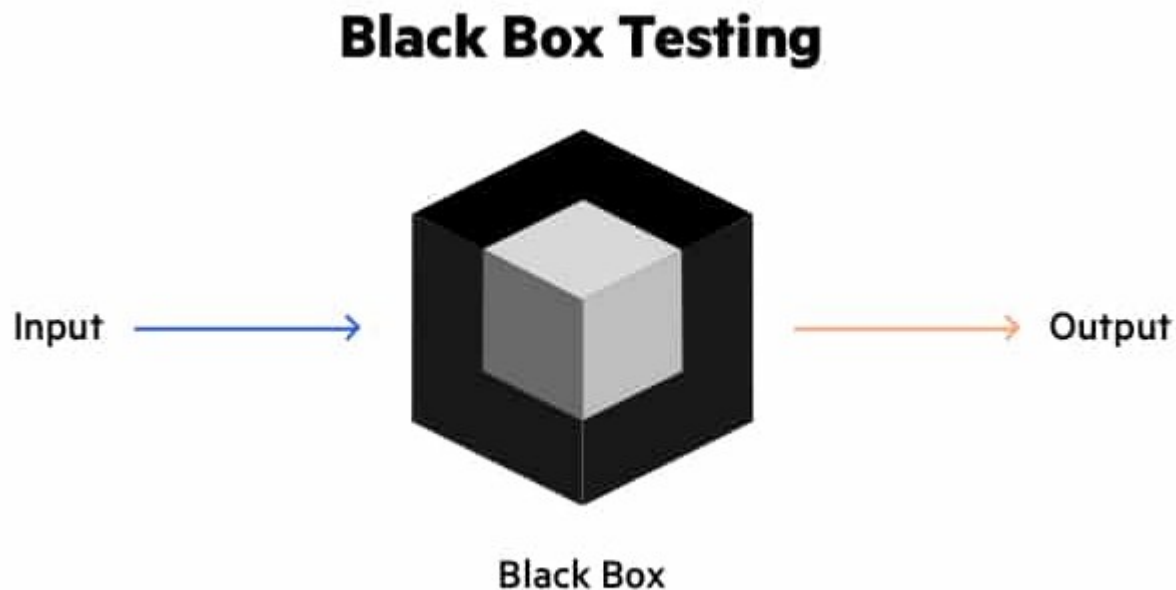
USER ACCOUNT FORM:

SL.No	Test Case	Excepted Result	Test Result
1	On the click of ADD button	At first user have to fill all fields with proper data , if any Error like entering text data instead of number or entering number instead of text..is found then it gives proper message otherwise Adds Record To the Database	successful
2.	On the Click of DELETE Button	This deletes the details of student by using Register no.	Successful
3.	On the Click of UPDATE Button	Modified records are Updated in database by clicking UPDATE button.	Successful
4.	On the Click of SEARCH Button	Displays the Details of book for entered Register no. Otherwise gives proper Error message.	Successful
5.	On the Click of CLEAR Button	Clears all fields	Successful
6.	On the Click of EXIT button	Exit the current book details form	successful
7.	On the Click of NEXT button	Display the next form	successful

BOOK RETURN FORM:

SL.No	Test Case	Excepted Result	Test Result
1	On the click of ADD button	At first user have to fill all fields with proper data , if any Error like entering text data instead of number or entering number instead of text..is found then it gives proper message otherwise Adds Record To the Database	successful
2.	On the Click of DELETE Button	Which deletes the details of book by using Register no.	Successful
3.	On the Click of UPDATE Button	Modified records are Updated in database by clicking UPDATE button.	Successful
4.	On the Click of SEARCH Button	Displays the Details of returned book ... Otherwise gives proper Error message.	Successful
5.	On the Click of CLEAR Button	Clears all fields	Successful
6.	On the Click of EXIT button	Exit the current book details form	successful
7.	On the Click of NEXT button	Display the next form	successful

Experiment 11: Prepare test cases for Black Box Testing



Types Of Black Box Testing

Black box testing can be applied to three main types of tests: functional, non-functional, and regression testing.

Functional Testing

Black box testing can test specific functions or features of the software under test. For example, checking that it is possible to log in using correct user credentials, and not possible to log in using wrong credentials.

Functional testing can focus on the most critical aspects of the software (smoke testing/sanity testing), on integration between key components (integration testing), or on the system as a whole (system testing).

Non-Functional Testing

Black box testing can check additional aspects of the software, beyond features and functionality. A non-functional test does not check “if” the software can perform a specific action but “how” it performs that action.

Black box tests can uncover if software is:

- Usable and easy to understand for its users
- Performant under expected or peak loads
- Compatible with relevant devices, screen sizes, browsers or operating systems
- Exposed to security vulnerabilities or common security threats

Regression Testing

Black box testing can be used to check if a new version of the software exhibits a regression, or degradation in capabilities, from one version to the next. Regression testing can be applied to functional aspects of the software (for example, a specific feature no longer works as expected in the new version), or non-functional aspects (for example, an operation that performed well is very slow in the new version).

Black Box Testing Techniques

Equivalence Partitioning

Testers can divide possible inputs into groups or “partitions”, and test only one example input from each group. For example, if a system requires a user’s birth date and provides the same response for all users under the age of 18, and a different response for users over 18, it is sufficient for testers to check one birth date in the “under 18” group and one date in the “over 18” group.

Boundary Value Analysis

Testers can identify that a system has a special response around a specific boundary value. For example, a specific field may accept only values between 0 and 99. Testers can focus on the boundary values (-1, 0, 99 and 100), to see if the system is accepting and rejecting inputs correctly.

Decision Table Testing

Many systems provide outputs based on a set of conditions. Testers can then identify “rules” which are a combination of conditions, identify the outcome of each rule, and design a test case for each rule.

For example, a health insurance company may provide different premium based on the age of the insured person (under 40 or over 40) and whether they are a smoker or not. This generates a decision table with four rules and up to four outcomes—below is an example with three possible outcomes.

Conditions	Rule 1	Rule 2	Rule 3	Rule 4
Under 40	False	False	True	True
Smoker	False	True	False	True

OUTCOMES

1: High premium				✓
2: Medium premium	✓	✓		
3: Low premium			✓	

In this case four use cases (one for each rule) would be sufficient to fully test the system.

